

DEPLOYMENT GUIDELINE 1 OF N • PLAIN-LANGUAGE WALKTHROUGH

Architecture & Environments — Explained

Your boss's team ran a deployment-readiness audit against the Core Health codebase. This document is `01-architecture-environments.md` — section 1 of that audit — translated into plain language, with a concrete, ordered plan for fixing everything it flagged.

SOURCE: `01-architecture-environments.md`

CHECKS: ARCH-01 .. ARCH-10

MUST-FIX ITEMS: 4

1 How to read the audit

Five words and two codes show up constantly in this document. Once these click, the rest reads itself.

PASS

This part works correctly. Nothing to do.

FAIL

This is missing or broken — confirmed by looking at the actual code.

PARTIAL

Part of it works, part doesn't. These are the trickiest — usually one good decision next to one gap.

CANNOT VERIFY

Not necessarily broken — the reviewer just couldn't find proof either way by reading the repository alone. Often means "undocumented," not "wrong."

N/A

Doesn't apply yet — the feature this check is about doesn't exist in the app yet, so there's nothing to check.

P0 / P1 / P2 (priority)

P0 = must fix before this can safely run in production. **P1** = should fix soon after. **P2** = fine to plan for later.

Effort: S / M / L

S = under a day. **M** = a few days. **L** = a week or more.

"Proof" column

The exact file and line number the reviewer read to reach that verdict — e.g. `server/server.ts:15-20`. You can open that file yourself and see exactly what they saw.

Out of the 10 checks in this section, **4 are P0 "must fix"**, all currently **FAIL** or **PARTIAL**. The rest are either **CANNOT VERIFY** (missing documentation, not necessarily a bug) or **N/A** (doesn't apply yet). Section 2 below walks through each of the 4 must-fix items.

2 The 4 things that must be fixed

Each one, in the order the audit lists it: what it actually says, why it matters in practice, and the concrete steps to close it.

ARCH-01 — No architecture diagram

P0

FAIL

effort: S

Nobody has drawn a picture of how Core Health's pieces fit together — which service talks to which, over what port, and where the database and file storage sit. Right now that understanding only exists by reading every file.

WHY IT MATTERS

A new developer — or your boss, or an auditor — can't get oriented in five minutes. They have to read code to answer a question a single picture would answer instantly.

HOW TO FIX IT

- 1 Create a file at `docs/architecture.md` in the repository.
- 2 Add one **Mermaid diagram** — a diagram written as plain text that GitHub renders automatically, no image file needed. It should show: Browser → Client (React) → API (Express + Socket.IO) → PostgreSQL, and → File storage.
- 3 Commit it.

ALREADY IN HAND

This is the *current*, as-deployed-today architecture — simpler and smaller than the AWS diagram already built in this conversation, which describes where things are *moving to*. Both are useful; this one just needs to exist first, and it's quick. Say the word and I'll write it directly into the repo.

ARCH-04 — Configuration & environment variables

P0

PARTIAL

effort: S

What already works: the server reads its settings from environment variables rather than hard-coded values — the right pattern. Three gaps sit next to that:

- 1 **Dev URLs are allowed in production.** `server.ts` and `config/socket.ts` both hard-code `http://localhost:5173/5174/5175` into the list of origins allowed to call the API — and that list is used in every environment, including production. It should only apply outside production.
- 2 **One frontend build only ever talks to one backend.** Vite "bakes" `VITE_API_URL` into the compiled JavaScript at build time (`client/Dockerfile:15-18`, `client/src/env.ts:5`) — so the Docker image you build for production can never be reused for staging, or anywhere else, without rebuilding it with a different URL baked in. The standard practice is "build once, deploy anywhere."
- 3 **Missing settings fail silently.** If `JWT_SECRET` or `DATABASE_URL` were ever missing, the server doesn't check for that at startup — it may start anyway and misbehave in a confusing way later, instead of refusing to start with a clear error immediately.

HOW TO FIX IT

- 1 Wrap the three `localhost` origins so they're only added when `process.env.NODE_ENV !== 'production'`.
- 2 Stop baking the API URL into the JavaScript bundle. Instead, have the frontend container write a small `config.js` file at startup (from an environment variable), and have the React app read the API URL from that file at runtime instead of from Vite's build-time constant. Same built image, any environment, no rebuild.
- 3 At the very top of `server.ts`, before anything else runs: list the environment variables the app cannot run without, check each one is actually set, and if any is missing, print a clear error and exit immediately rather than limping along.

WHY IT MATTERS

A missing secret that fails silently is far harder to debug at 2am in production than one that stops the server immediately with a clear message pointing at exactly what's missing.

ARCH-05 — Running more than one server at once

P0

PARTIAL

effort: M

What already works: login uses JWTs (a signed token the browser holds), not a session stored in server memory — so users stay logged in correctly even if the server restarts, and this part is already safe to run on multiple instances.

What isn't ready yet, for the moment you run **2 or more** server copies side by side (exactly what the AWS plan calls for):

- 1 **Uploaded files live on local disk** — a file uploaded through instance A simply doesn't exist on instance B. (This is ARCH-06, below — the same root cause.)
- 2 **Live notifications (Socket.IO) have no shared memory between instances.** If a user is connected to instance A, and something happens that should notify them while the server processes it on instance B, that notification never reaches them — the two instances don't know about each other's connected users at all.

HOW TO FIX IT

- 1 Fix the uploads problem first (ARCH-06 below) — do that before this one.
- 2 Add the `@socket.io/redis-adapter` package (plus a Redis client, e.g. `ioredis`).
- 3 Stand up a small Redis instance — AWS ElastiCache is the natural choice in the AWS plan already built.
- 4 In `config/socket.ts`, right after creating the `io` server, attach the adapter:
`io.adapter(createAdapter(pubClient, subClient))`. This makes every server instance share connection state through Redis, so a notification reaches the right user no matter which instance they happen to be connected to.

DIRECTLY AFFECTS THE AWS PLAN

The AWS architecture already built in this conversation runs

2 EC2 INSTANCES

behind a load balancer. Without this fix, that setup would silently drop notifications for roughly half your users, depending on which instance the load balancer happened to route them to. This needs to be done *before* going live on that 2-instance setup, not after.

ARCH-06 — Where uploaded files are stored

P0

FAIL

effort: M

Prescription files, lab reports, and patient report uploads are all saved into a folder that lives inside the running server process (`server.ts:30-34`, and the routes that handle each upload type). The project's own deploy notes say plainly: these files get deleted every time the server restarts or redeploys.

WHY IT MATTERS

This isn't a "what happens if we scale to 2 instances" risk — it's already live data loss today, with a single instance, on every redeploy. It's the most urgent item on this whole list.

HOW TO FIX IT

- 1 Create a private S3 bucket (already part of the AWS plan).
- 2 Replace local-disk storage with `multer-s3` (or stream uploads to S3 directly) in the three upload routes — prescriptions, lab reports, patient reports.
- 3 Replace `express.static('/uploads')` with a route that hands out short-lived, signed S3 links to authorized users only, instead of serving files directly off disk.
- 4 Before switching over, run a one-time script to copy any files you still need from local disk up to the new bucket.

3 The rest of the list

Nothing below is marked FAIL or PARTIAL — these are either "we couldn't tell from the code" or "doesn't apply yet." None of them block going live.

ID	PRIORITY	STATUS	WHAT IT ACTUALLY MEANS
ARCH-02	P0	CANNOT VERIFY	The repo only describes one environment, called "production" (railway.toml:1). No separate staging/test environment is documented anywhere. Worth fixing — see §4.
ARCH-03	P0	CANNOT VERIFY	Depends entirely on ARCH-02 — can't check how environments differ if there's only one.
ARCH-07	P1	CANNOT VERIFY	No written list exists of "what's the one thing that, if it breaks, takes the whole app down." Worth a short written note — the AWS diagram already shows some obvious ones (the single NAT Gateway, for example).
ARCH-08	P1	N/A	This check is about how the app handles outside services (email, SMS, payments). You don't call any yet — nothing in server/package.json does that. This becomes relevant the moment one is added, not before.
ARCH-09	P1	CANNOT VERIFY	Needs an actual load test (simulating many users at once) to answer. A later-stage task, not a blocker.
ARCH-10	P2	CANNOT VERIFY	Needs a capacity plan (how much traffic can this handle before it falls over). Also a later-stage task.

4 Suggested order of work

A few of these depend on each other — this order avoids doing something twice.

- 1 **ARCH-01 — write the architecture diagram.** Quick, independent, do it first so everyone's looking at the same picture while the rest gets fixed.
- 2 **ARCH-04 — environment variable fixes.** Independent, small, foundational — do it early.
- 3 **ARCH-06 — move uploads to S3.** Do this before ARCH-05, since the Socket.IO fix below assumes files are no longer a local-instance problem.
- 4 **ARCH-05 — add the Redis adapter for Socket.IO.** Now safe to tackle, with uploads already off local disk.
- 5 **Stand up a real staging environment on AWS** — a second, smaller copy of the same stack (network, compute, data) built in this conversation, separate from production. This resolves ARCH-02 and ARCH-03 as a natural side effect of the AWS migration, rather than a separate task.
- 6 **ARCH-07 — write the single-points-of-failure note.** Can be a short appendix to the architecture diagram from step 1, once the AWS plan's own weak points (the shared NAT Gateway, Single-AZ RDS if you keep it that way) are factored in.
- 7 **ARCH-08, 09, 10 — leave for later.** Not blockers; revisit once there's real traffic or a reason to add an outside service.

5 How this connects to the AWS plan

ALREADY FLAGGED, NOW CONFIRMED

ARCH-06 (uploads on local disk) is the exact same gap already called out as a hard blocker in the AWS deployment architecture document from this conversation — your boss's audit independently caught the same issue. It's no longer just a suggestion; it's an official P0 finding.

A NEW HARD REQUIREMENT FOR THE AWS PLAN

ARCH-05 (the Socket.IO Redis adapter) wasn't called out before, but it now must happen before the AWS setup goes live — that plan runs 2 EC2 instances specifically for redundancy, and without this fix, live notifications would silently fail for whichever users land on the "wrong" instance.

Practically: treat ARCH-06 and ARCH-05 as part of the AWS migration itself, not separate follow-up work — build order §9 in the AWS guideline (the Terraform build order) should have these two done before step 13, "smoke-test the real domain end to end."

Core Health — plain-language walkthrough of 01-architecture-environments.md. Source document unchanged; this is an explanation and action plan built from it.